

Dense Passage Retrieval for Open-Domain Question Answering

Nguyen Phu Tai, Le Duong Minh Thien, Vo Dinh Trung, Vu Anh Tuan

[†]University of Information Technology

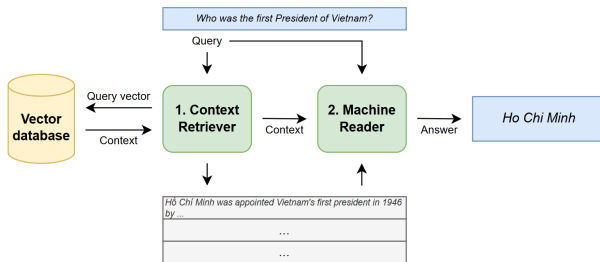
{22521280, 22521386, 22521571, 22521614}@gm.uit.edu.vn

Abstract

Trong bài toán **Question Answering (QA)**, đặc biệt là Open-Domain QA, việc truy xuất đoạn văn là bước quan trọng để tìm ra các đoạn văn phù hợp nhằm trả lời câu hỏi. Trước đây, các phương pháp sử dụng vector thưa (sparse vector) như TF-IDF hoặc BM25 thường được áp dụng nhờ tính hiệu quả và tốc độ xử lý. Trong dự án này, chúng tôi chứng minh rằng việc sử dụng vector đặc (dense vector) có thể cải thiện đáng kể hiệu quả truy xuất trong thực tế. Phương pháp này tận dụng các embedding được tạo ra từ một mô hình đơn giản, sử dụng kiến trúc **dual-encoder**, đạt hiệu quả cao ngay cả khi sử dụng lượng dữ liệu huấn luyện hạn chế. Từ đó, tạo nên một mô hình truy xuất đoạn văn sử dụng dense vector hiệu quả, hay còn gọi là **Dense Passage Retrieval (DPR)**.

1 Giới thiệu

Open-Domain Question Answering (ODQA) là bài toán trả lời các câu hỏi thực tế dựa trên một (hoặc nhiều) tập tài liệu lớn. So với các cách tiếp cận truyền thống, phương pháp hiện đại thường được chia thành hai giai đoạn chính. Thứ nhất, **Context Retriever** truy xuất một lượng nhỏ các đoạn văn có khả năng chứa đáp án. Thứ hai, **Machine Reader** phân tích và đánh giá các đoạn văn này để xác định câu trả lời chính xác.



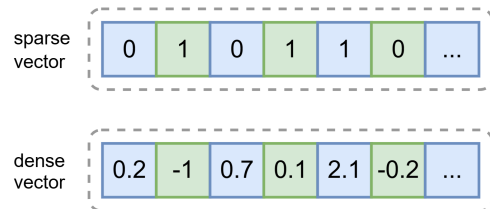
Ảnh 1: Kiến trúc hiện đại cho bài toán ODQA.

1.1 Sự khác biệt giữa sparse vector và dense vector

Trong các hệ thống truyền thống, câu hỏi và ngữ cảnh được biểu diễn bằng **sparse vector** dựa trên tần suất và trọng số từ khóa (TF-IDF, BM25). Những phương pháp này tuy hiệu quả trong tìm kiếm dựa trên từ khóa nhưng có hạn chế: Phụ thuộc hoàn toàn vào từ khóa. Nó sẽ chỉ hiệu quả khi các từ khóa trong câu hỏi và tài liệu trùng khớp hoàn toàn, khó nhận diện được từ đồng nghĩa hay các cách diễn đạt khác nhau, không nắm bắt được mối liên hệ ngữ nghĩa sâu sắc trong câu.

Trái lại, **dense vector** tận dụng các mô hình học sâu như BERT, GPT, hoặc phương pháp nhúng từ như Word2Vec để mã hóa ngữ nghĩa phức tạp. Cho phép hệ thống có thể nhận diện mối liên hệ giữa các từ đồng nghĩa hoặc cách diễn đạt khác nhau, ví dụ: từ "kẻ xấu" trong câu hỏi có thể được liên kết với từ "phản diện" trong đoạn văn. Điều này vượt trội hơn các phương pháp dựa vào từ khóa.

Ngoài ra, dense vector hoạt động trong không gian liên tục, đem lại sự linh hoạt hơn sparse vector trong biểu diễn. Đồng thời, các embedding function cũng có thể được tối ưu cho các mục tiêu cụ thể như suy luận hoặc nhận diện thông tin liên quan.



Ảnh 2: So sánh giữa sparse vector và dense vector.

1.2 Phương pháp tiếp cận

Trong dự án này, chúng tôi đặt ra và giải quyết câu hỏi: "Liệu có thể huấn luyện một mô hình dense embedding hiệu quả chỉ với các cặp câu hỏi - đoạn

văn hay không?”.

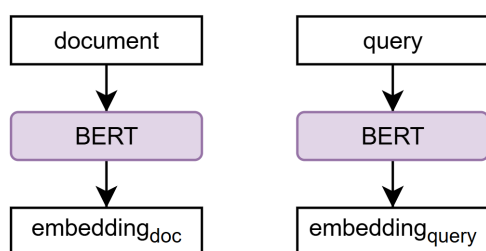
Để chứng thực, chúng tôi sử dụng mô hình BERT được tiền huấn luyện, và áp dụng kiến trúc dual-encoder. Quá trình huấn luyện sẽ tập trung tối ưu hoá các embedding để tối đa hóa tích vô hướng (inner product) giữa vector của câu hỏi và đoạn văn liên quan. Với mục tiêu: Trong mỗi batch dữ liệu, tất cả các cặp câu hỏi - đoạn văn sẽ được so sánh, nhằm giúp mô hình học cách phân biệt các đoạn văn phù hợp và không phù hợp.

Kết luận rút ra là: Với chiến lược huấn luyện hợp lý, không cần bước tiền huấn luyện phức tạp, chỉ cần tinh chỉnh các bộ mã hóa (encoder) câu hỏi và đoạn văn trên dữ liệu hiện có là đủ để tăng tính hiệu quả của mô hình. Và trong bối cảnh ODQA, độ chính xác trong truy xuất đoạn văn được cải thiện cũng dẫn đến việc độ chính xác của hệ thống QA tổng thể sẽ được nâng cao.

1.3 Tại sao lại sử dụng dual-encoder thay vì bi-encoder?

Chúng tôi sẽ tiến hành so sánh giữa hai loại kiến trúc trong trường hợp chúng được áp dụng vào bài toán này, và chỉ ra loại kiến trúc tối ưu hơn.

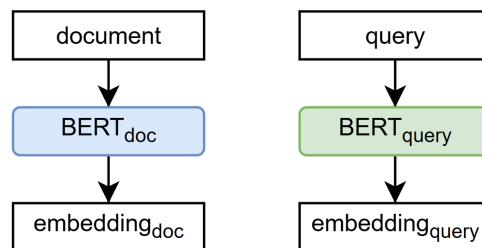
Đầu tiên, **bi-encoder** sử dụng cùng một encoder để mã hóa cả câu hỏi và đoạn văn, tức là cùng một bộ trọng số cho cả hai đầu vào. Mặc dù có ưu điểm tiết kiệm tham số, dẫn đến mô hình nhỏ gọn hơn, dễ bảo trì. Tuy nhiên cũng vì thế mà nó không đủ linh hoạt để nắm bắt các đặc trưng riêng biệt của câu hỏi và đoạn văn, dẫn đến giảm hiệu quả trong các bài toán phức tạp.



Ảnh 3: Kiến trúc bi-encoder.

Ngược lại, **dual-encoder** sử dụng hai encoder riêng biệt cho câu hỏi và đoạn văn. Có thể có cùng kiến trúc, nhưng trọng số sẽ khác nhau. Điều này cho nó khả năng tối ưu hóa riêng biệt cho từng loại đầu vào, phù hợp hơn với đặc thù ngôn ngữ của chúng. Như câu hỏi (query) thường có cấu trúc ngắn, đôi khi bao hàm thực thể (entity) hoặc từ nghi vấn. Còn đoạn văn (passage) thường có cấu trúc dài, chứa nhiều thông tin. Dù tốn tài nguyên

hơn, dual-encoder lại có khả năng học cách nắm bắt ngữ cảnh, phân tách thông tin cốt lõi để có thể liên kết được với nội dung mà query muốn biết.



Ảnh 4: Kiến trúc dual-encoder.

Nhìn chung, cả hai loại kiến trúc đều cùng dựa trên cơ chế “hai đầu vào - hai embedding” rồi so sánh vector, chỉ khác ở chỗ chúng có dùng chung tham số hay không. Dựa vào quy mô của bài toán ODQA đang xét, và các đặc tính đã phân tích như trên, chúng tôi cho rằng dual-encoder là tối ưu hơn.

2 Bộ dữ liệu

Trong dự án này, chúng tôi sử dụng hai nguồn dữ liệu chính để xây dựng và huấn luyện hệ thống DPR, bao gồm:

Wikipedia: Là nguồn dữ liệu cơ sở, chứa các đoạn văn được trích xuất từ bản dump của English Wikipedia (20/12/2018). Dữ liệu trích xuất này đã được chuẩn hóa và phân chia thành các khối văn bản rời rạc dài 100 từ mỗi khối, đóng vai trò là đoạn văn - đơn vị truy xuất cơ bản.

Natural Questions (NQ): Bộ dữ liệu hỏi-đáp (QA) được thiết kế đặc biệt cho bài toán Open-Domain QA, cung cấp bởi Google, chứa các câu hỏi thực tế cùng câu trả lời và ngữ cảnh liên quan, được gắn nhãn bởi những người chú thích trên Wikipedia.

2.1 Cấu trúc của dữ liệu

Wikipedia: Lưu ở dạng file .tsv, gồm ba cột chính:

- **id:** ID duy nhất của mỗi đoạn văn.
- **text:** Nội dung đoạn văn.
- **title:** Tiêu đề bài viết Wikipedia chứa đoạn văn.

Natural Questions (train, dev): Lưu ở dạng file .json, chứa các đối tượng có cấu trúc:

- **dataset:** Tên tập dữ liệu.
- **question:** Câu hỏi.
- **answers:** Danh sách các câu trả lời đúng.

- **positive_ctxs**: Danh sách các đoạn văn chứa câu trả lời (*positive passage*).
- **negative_ctxs**: Danh sách các đoạn văn không chứa câu trả lời (*negative passage*).
- **hard_negative_ctxs**: Danh sách các đoạn văn khó, không chứa câu trả lời nhưng có liên quan đến câu hỏi qua từ khóa (khớp nhiều từ khóa) (*hard negative*).

Natural Questions (test): Lưu ở dạng file .csv, không có header, gồm hai cột chính:

- **Cột 1**: Câu hỏi.
- **Cột 2**: Danh sách chứa một hoặc nhiều đáp án.

2.2 Tiền xử lý dữ liệu

Dữ liệu sử dụng sẽ được tiền xử lý trước để đảm bảo tính hiệu quả khi đưa vào mô hình, cụ thể như sau:

Dữ liệu Wikipedia: Đầu tiên, loại bỏ dữ liệu bán cấu trúc có trong văn bản như: bảng, biểu đồ, danh sách, trang điều hướng, ... Sau đó chia văn bản thành các đoạn văn cố định với 100 từ mỗi đoạn. Tiêu đề bài viết tương ứng sẽ được thêm vào đầu mỗi đoạn văn, và được ngăn cách với đoạn bằng ký tự [SEP].

Dữ liệu Natural Questions: Câu trả lời và ngữ cảnh được ánh xạ chính xác trong phiên bản Wikipedia được sử dụng bởi người chú thích. Các câu hỏi không thể tìm thấy ngữ cảnh hoặc đáp án sẽ bị loại bỏ.

Loại dữ liệu	Tập tin	Số lượng
Wikipedia	psgs_w100.tsv	21015324
NQ (train)	nq-train.json	58880
NQ (validate)	nq-dev.json	6515
NQ (test)	nq-test.csv	3609

Bảng 1: Quy mô dữ liệu sử dụng trong dự án.

3 Phương pháp luận

Mục tiêu cuối cùng: Xây dựng và tối ưu hóa bộ truy vấn $R : (q, C) \rightarrow C_F$, là một hàm nhận đầu vào là câu hỏi q và tập ngữ liệu C , đầu ra là tập con $C_F \in C (|C_F| = k \ll |C|)$ chứa k đoạn văn có khả năng liên quan cao nhất trong C cho câu hỏi q .

3.1 Dense Encoder

Hệ thống DPR sử dụng kiến trúc dual-encoder với hai encoder độc lập: Một encoder cho đoạn văn

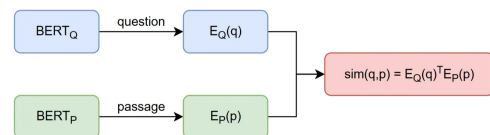
(E_P) và một encoder cho câu hỏi (E_Q), được triển khai dựa trên hai mạng BERT (base, uncased) độc lập. Chúng sẽ mã hóa đầu vào tương ứng của chúng thành các dense vector trong không gian vector $\mathbb{R}^d$.

Về cách thức mã hóa, các encoder sẽ mã hóa đầu vào của chúng theo luồng sau đây:

1. Sử dụng tokenizer để tách đoạn văn/câu hỏi thành các token.
2. Thêm các token đặc biệt: [CLS] ở đầu câu và [SEP] ở cuối câu để xác định ranh giới câu/đoạn. Token [CLS] đóng vai trò nhận dạng toàn câu đầu vào (được BERT dùng để sinh ra embedding cho toàn câu). Còn token [SEP] giúp phân tách câu/đoạn khi có nhiều đoạn, hoặc chỉ làm tín hiệu kết thúc câu.
3. Mã hóa qua BERT: Ánh xạ các token thành các vector embedding rồi truyền qua các lớp Transformer. Cuối cùng, ta thu được một chuỗi vector ẩn (hidden state), mỗi vector ứng với một token (bao gồm cả [CLS]).
4. Lấy hidden state của vị trí token [CLS] ở layer cuối cùng làm vector đại diện cho toàn bộ đoạn văn/câu hỏi.

Trong quá trình suy luận, đầu tiên, tất cả các đoạn văn sẽ được mã hóa trước bằng encoder E_P , và lập chỉ mục bằng FAISS (một thư viện tối ưu hóa cho tìm kiếm vector và phân nhóm hiệu quả trên quy mô lớn). Sau đây, với mỗi câu hỏi q , hệ thống tính embedding $v_q = E_Q(q)$ của nó trong thời gian thực và truy xuất k đoạn văn có điểm tương đồng cao nhất với v_q qua inner product của chúng:

$$\text{sim}(q, p) = E_Q(q)^T E_P(p) \quad (1)$$



Ảnh 5: Quá trình xử lý và tính toán embedding.

Tích vô hướng (inner product) được sử dụng vì tính đơn giản của nó, và khi so sánh với các phương pháp tính độ tương đồng khác như cosine hoặc L2 thì không có sự khác biệt quá lớn.

3.2 Hàm Loss

Cho $D = \left\{ \langle q_i, p_i^+, p_{i,1}^-, \dots, p_{i,n}^- \rangle \right\}_{i=1}^m$ là dữ liệu huấn luyện chứa m mẫu, mỗi mẫu chứa một câu hỏi q_i , một đoạn văn chứa đáp án (positive passage) p_i^+ và n đoạn văn không chứa đáp án (negative passage) $p_{i,j}^-$. Mục tiêu là tăng độ tương đồng giữa q_i và p_i^+ , đồng thời giảm độ tương đồng giữa q_i và các $p_{i,j}^-$ ($i \neq j$). Để làm điều này, cần tối ưu hàm negative log likelihood:

$$L(q_i, p_i^+, p_{i,1}^-, \dots, p_{i,n}^-) = -\log \frac{e^{\text{sim}(q_i, p_i^+)}}{e^{\text{sim}(q_i, p_i^+)} + \sum_{j=1}^n e^{\text{sim}(q_i, p_{i,j}^-)}} \quad (2)$$

Trong đó:

- $\text{sim}(\cdot)$: Phép đo độ tương đồng giữa embedding của hai đối tượng (câu hỏi/đoạn văn), sử dụng inner product.
- Số mũ $e^{\text{sim}(q_i, p_i^+)}$: Biểu thị mức “ưu tiên” của positive passage.
- Phép chia $(e^{\text{sim}(q_i, p_i^+)} + \sum_{j=1}^n e^{\text{sim}(q_i, p_{i,j}^-)})$: Áp đặt cơ chế softmax, biến điểm tương đồng thành xác suất tương đối so với các negative passages.
- Hàm log: Được dùng trong negative log likelihood, giúp tối ưu dễ dàng hơn, giảm hiện tượng overflow (thường dùng trong các mô hình phân loại).

Về bản chất, khi mô hình học, hàm sẽ khuyến khích $\text{sim}(q_i, p_i^+)$ tăng lên, đồng thời giảm $\text{sim}(q_i, p_{i,j}^-)$. Ta sẽ phạt mô hình (bằng việc tăng giá trị loss) nếu positive passage chưa đủ “nổi trội” hơn các negative passage.

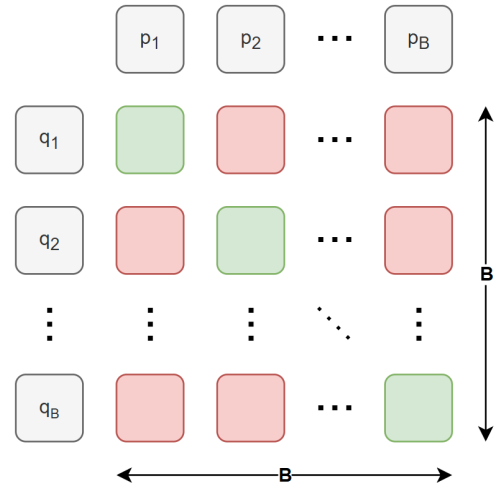
Sau đây là phân tích rõ hơn về nguyên lý hoạt động: Đầu tiên, áp dụng softmax trên toàn bộ các đoạn văn (1 positive + n negative) để tính xác suất cho mỗi đoạn. Bằng cách lấy negative log của xác suất positive, mô hình được dẫn dắt đến trạng thái mà positive passage có xác suất cao nhất (tương ứng với độ tương đồng tối đa). Kết quả là mô hình dual-encoder học được cách ánh xạ embedding của q_i và p_i^+ lại gần nhau trong không gian vector, đồng thời đẩy xa các negative passage.

3.3 Chuẩn bị cho quá trình huấn luyện

Về dữ liệu, sẽ là sự kết hợp giữa positive passage và negative passage: Positive passage là những đoạn

chứa đáp án đã được gắn nhãn sẵn trong bộ dữ liệu Natural Questions. Còn negative passage sẽ kết hợp hard negative passage từ BM25 và các positive passage của những câu hỏi khác trong cùng batch (gọi là Gold).

Sau đó, tiến hành huấn luyện sử dụng phương pháp **in-batch negatives**. Phương pháp này tận dụng các đoạn văn trong cùng một batch để tăng số mẫu huấn luyện và tái sử dụng các phép tính toán. Tức là, trong một batch có kích thước B , cũng là một ma trận điểm tương đồng kích thước $B \times B$, mỗi câu hỏi sẽ được ghép với $B - 1$ đoạn văn khác để tạo ra các cặp negative. Đồng thời, mỗi câu hỏi cũng được ghép với positive passage của nó.



Ảnh 6: Mô hình phương pháp in-batch negative.

3.4 Phương pháp đánh giá: Top-k Accuracy

Top-k accuracy là tỷ lệ các câu hỏi mà trong đó tập k đoạn văn được truy vấn sẽ có ít nhất một đoạn chứa đáp án đúng. Đây là thước đo đánh giá khả năng bao phủ các tài liệu chứa câu trả lời trong tập kết quả đầu tiên của hệ thống.

$$\text{Top-k accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}_{\{C_F^i \cap C_{\text{gold}}^i \neq \emptyset\}} \quad (3)$$

Trong đó:

- N : Tổng số câu hỏi trong tập kiểm tra.
- C_F^i : Tập k đoạn văn được truy xuất cho câu hỏi i .
- C_{gold}^i : Tập các đoạn văn chứa đáp án đúng cho câu hỏi i .

- $\mathbb{K} \left\{ C_F^i \cap C_{\text{gold}}^i \neq \emptyset \right\}$: Hàm chỉ thị, trả về 1 nếu điều kiện bên trong đúng (tức là trong số các đoạn văn truy xuất được có tồn tại đoạn văn có chứa đáp án đúng), ngược lại trả về 0.
- Lưu ý: Nếu tại top-k có tồn tại đoạn văn chứa đáp án đúng, thì tại top-h (với $h > k$) cũng sẽ tồn tại đoạn văn chứa đáp án đúng.

Chỉ số này cho biết mức độ hệ thống có thể tìm thấy các đoạn văn chứa đáp án chính xác trong top k kết quả được truy xuất, tức là đo lường khả năng bao phủ của hệ thống. Điều này rất quan trọng trong bài toán ODQA khi giai đoạn sau (Machine Reader) chỉ xử lý các đoạn văn trong danh sách này. Ngoài ra, nó còn là một chỉ số trực quan và đơn giản, thường được dùng để so sánh hiệu suất giữa các phương pháp như BM25 và DPR.

4 Cài đặt thí nghiệm

4.1 Thông tin cấu hình

Mô hình sẽ được thiết lập trên cấu hình jupyter notebook được cung cấp bởi Kaggle. Cụ thể như sau:

- Ngôn ngữ lập trình: Python 3.10
- Thư viện chính:
 - PyTorch (huấn luyện và suy luận mô hình, quản lý GPU).
 - Hugging Face Transformers (sử dụng mô hình BERT và tokenizer).
 - FAISS (tìm kiếm vector hiệu quả trong quy mô lớn).
 - Pandas, NumPy, scikit-learn, Matplotlib, Seaborn (xử lý dữ liệu và trực quan hóa dữ liệu).
- Phần cứng: 2 GPU Tesla T4, 16 GB VRAM/GPU.

4.2 Chi tiết triển khai

Mô hình DPR được triển khai sẽ là mô hình đã được huấn luyện sẵn trên bộ dữ liệu Natural Questions, nó sử dụng hai mô hình BERT (base, uncased) riêng biệt cho document encoder và query encoder. Cụ thể như sau:

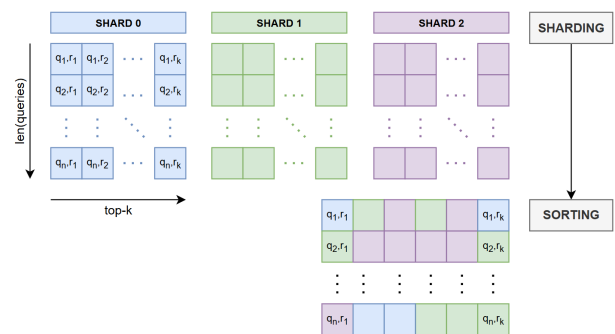
Đầu tiên, tải trọng số mô hình đã được huấn luyện từ trước và khởi tạo mô hình DPR. Sau đó, thiết lập các thông số cần thiết cho việc triển khai mô hình như:

- Batch size khi mã hóa: 256.

- Chiều của vector embedding: 768 (dựa vào phiên bản BERT base).
- Chunk size cho việc đọc dữ liệu lớn (Wikipedia): 10^6 đoạn văn/shard.
- Số shard: $\left\lceil \frac{21\,015\,324}{10^6} \right\rceil \approx 22$.

Sau khi đã thiết lập hoàn tất, ta tiến hành xử lý dữ liệu và mã hóa chúng: Đọc và nạp dữ liệu Wikipedia và dữ liệu của tập Natural Questions dùng cho việc kiểm tra, trong đó dữ liệu Wikipedia sẽ được nạp theo từng khối nhỏ (chunk), lưu thành các shard để tránh quá tải bộ nhớ. Mỗi loại dữ liệu sẽ được đưa vào encoder tương ứng trong mô hình đã huấn luyện. Sử dụng tokenizer từ BERT với max_length=256, cho phép tự động padding và truncation để nhất quán các token được tạo ra. Vector cuối cùng của đoạn văn/câu hỏi tương ứng từ mỗi loại dữ liệu sẽ được lấy từ hidden state của token [CLS].

Tiếp đến, xây dựng chỉ mục (index) cho dữ liệu Wikipedia bằng FAISS và truy xuất dữ liệu. Tuy nhiên, sau khi mã hóa, bởi vì kích thước quá lớn của dữ liệu Wikipedia, nó sẽ được chia thành từng shard tương tự như khi nạp dữ liệu. Sau đó mới tiến hành xây dựng chỉ mục, sử dụng inner product IndexFlatIP. Ứng với mỗi chỉ mục riêng của từng shard, tiến hành truy vấn và tìm ra top-k tài liệu. Kết quả cuối cùng sẽ là sự hợp nhất tài liệu giữa tất cả các shard, sắp xếp lại theo điểm tương đồng giảm dần và chọn lại ra top-k tài liệu có điểm tương đồng lớn nhất ở mỗi câu hỏi.



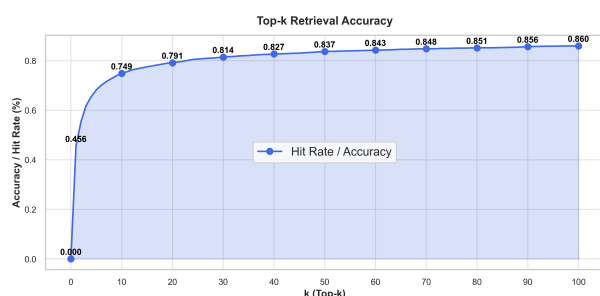
Ảnh 7: Minh họa quá trình sharding và sorting trong bước truy vấn.

Cuối cùng, với mỗi câu hỏi, hệ thống sẽ truy xuất top-100 đoạn văn có giá trị inner product lớn nhất. Tiến hành kiểm tra xem trong các top-k ($k=1, \dots, 100$) có chứa đoạn văn nào mà có từ khóa khớp với từ khóa trong câu hỏi hay không. Từ đó tiến hành tính top-k accuracy dựa vào công thức (3).

5 Kết quả thí nghiệm

5.1 Trực quan kết quả

Sau đây là biểu đồ biểu thị đường cong thể hiện xu hướng bao phủ của hệ thống DPR đã huấn luyện và kiểm tra, sau khi tiến hành truy xuất trên top-100 đoạn văn (dựa trên inner product) cho mỗi câu hỏi trong tập kiểm tra của bộ dữ liệu Natural Questions.



Ảnh 8: Biểu đồ trực quan hóa khả năng bao phủ của hệ thống DPR đã huấn luyện.

Tăng nhanh tại k=1-10: Khi k=1, Top-1 Accuracy đạt 46%, nghĩa là gần một nửa câu hỏi đã tìm được đáp án đúng chỉ với một đoạn văn. Khi tăng lên k=10, Accuracy tăng vọt lên 75%, cho thấy việc bổ sung thêm chỉ vài đoạn văn đã cải thiện đáng kể khả năng tìm đáp án.

Tăng chậm từ k=10-50 và bão hòa tại k>50: Trong tác vụ truy vấn quy mô lớn, số kết quả cần xem xét để “đảm bảo” đáp án thường tăng nhanh ở giai đoạn đầu. Để “nhặt” thêm những trường hợp hóc búa, phải “đào sâu” vào hàng chục (hoặc hơn) kết quả, dẫn đến tốc độ cải thiện chậm và sự bão hòa của đường cong.

Tác động đến hệ thống QA kế tiếp: Đối với dạng bài toán ODQA, việc dừng ở top-10 hay top-20 là phổ biến và đã đủ, vì bước kế tiếp (Machine Reader) chỉ cần một số đoạn văn để nhận dạng đáp án. Từ biểu đồ, ta thấy ngay cả top-20 đã đạt 79%-80%, và top-50 tiến sát 84%-85%. Tùy nhu cầu thực tế mà ta lựa chọn k sao cho cân bằng giữa tốc độ và độ chính xác.

5.2 Hướng cải thiện

Sau khi hoàn thiện, nhóm đã thực hiện tìm hiểu và đưa ra được vài phương hướng cải thiện để có thể nâng cao hơn nữa chất lượng mô hình DPR trong tương lai.

Tăng số lượng mẫu hard negative: Tăng cường những đoạn văn nhiễu tuy sẽ gây khó khăn cho mô hình hơn khi học, nhưng nếu học đủ tốt thì nó sẽ

giúp mô hình phân biệt chính xác hơn.

Cross-Encoder Re-ranking: Sau khi lấy top-k bằng dual-encoder, có thể dùng mô hình cross-encoder (hay re-ranker) để sắp xếp lại top-k với độ chính xác cao hơn, nâng tỷ lệ tìm thấy đáp án ở danh sách đoạn văn trả về bởi hệ truy vấn.

Sử dụng mô hình lớn hơn: Sử dụng BERT-large hoặc RoBERTa để có thể tạo ra các embedding có chất lượng cao hơn, đẩy đường con của biểu đồ tăng sớm hoặc bão hòa ở mức cao hơn.

References

- [1] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering". [arXiv:2004.04906](https://arxiv.org/abs/2004.04906).
- [2] facebookresearch, github.com/facebookresearch/DPR. "DPR".
- [3] Hannibal046, github.com/Hannibal046/nanoDPR. "nanoDPR".
- [4] IR with PUGGY, "DPR: Dense Passage Retrieval for Open-Domain Question Answering". youtu.be/TbObc9s8Gzw.